

Day 1 – Code Snippets: Xunit v3 & Fluent Assertions

Each step builds on the previous one.

Run `dotnet test` after each step to see tests go green.

 Full source: `Day1\Solutions\Lecture\Day1.Lecture.sln`

Step 1 – The Class Under Test (no tests yet)

Start with a plain C# class. Nothing special – just the business logic we want to protect.

```
//  Day1.Calculator/Calculator.cs
namespace Day1.Calculator;

public class Calculator
{
    public int Add(int a, int b) => a + b;

    public int Subtract(int a, int b) => a - b;

    public int Multiply(int a, int b) => a * b;

    // Special case: throws when b is zero
    public int Divide(int a, int b)
    {
        if (b == 0)
            throw new DivideByZeroException("Cannot divide by zero.");

        return a / b;
    }

    public bool IsEven(int number) => number % 2 == 0;
}
```

The class has no test awareness – it's pure business logic.

Step 2 – First [Fact] with Native Assert

Create a test project, add the class under test as a reference, write the first test.

```
// 📁 Day1.Calculator.Tests/CalculatorTests.cs
using Day1.Calculator;
using Xunit;

namespace Day1.Calculator.Tests;

public class CalculatorTests
{
    // Constructor = Arrange shared state (runs before every test)
    private readonly Calculator _calculator;

    public CalculatorTests()
    {
        _calculator = new Calculator();
    }

    // [Fact] – runs once, no parameters
    [Fact]
    public void Add_TwoPositiveNumbers_ReturnsCorrectSum()
    {
        // Arrange
        int a = 3, b = 5;

        // Act
        int result = _calculator.Add(a, b);

        // Assert – Expected first, then actual (XUnit convention)
        Assert.Equal(8, result);
    }

    [Fact]
    public void IsEven_EvenNumber_ReturnsTrue()
    {
        bool result = _calculator.IsEven(8);
        Assert.True(result);    // Assert.True / Assert.False for booleans
    }
}
```

```
# Run from the test project folder
dotnet test
# Output: Passed: 2
```

Step 3 – [Theory] + [InlineData]

Same test, different inputs – without duplicating the test body.

```
// 📁 Day1.Calculator.Tests/CalculatorTests.cs (continued)

// [Theory] marks a parameterized test
// Each [InlineData] row is one test entry in Test Explorer
[Theory]
[InlineData(10, 2, 5)] // 10 / 2 = 5
[InlineData(9, 3, 3)] // 9 / 3 = 3
[InlineData(0, 5, 0)] // 0 / 5 = 0
[InlineData(-8, 2, -4)] // -8 / 2 = -4
public void Divide_ValidDivisor_ReturnsCorrectQuotient(
    int a, int b, int expected) // parameters match [InlineData] order
{
    int result = _calculator.Divide(a, b);
    Assert.Equal(expected, result);
}

// Boolean theories work the same way
[Theory]
[InlineData(2, true)]
[InlineData(100, true)]
[InlineData(3, false)]
[InlineData(99, false)]
public void IsEven_VariousNumbers_ReturnsExpectedBool(int number, bool expected)
{
    Assert.Equal(expected, _calculator.IsEven(number));
}
```

Test Explorer shows 4 + 4 separate rows – each can fail independently.

Step 4 – Exception Testing with `Assert.Throws<T>`

When the expected outcome **is** an exception, use `Assert.Throws`.

```
// 📁 Day1.Calculator.Tests/CalculatorTests.cs (continued)

[Fact]
public void Divide_ByZero_ThrowsDivideByZeroException()
{
    // Assert.Throws runs the lambda and:
    //   ✓ passes if the exception is thrown
    //   ✗ fails if no exception or a different exception is thrown
    var ex = Assert.Throws<DivideByZeroException>(
        () => _calculator.Divide(10, 0));

    // The returned exception lets you assert on its message
    Assert.Contains("zero", ex.Message,
        StringComparison.OrdinalIgnoreCase);
}

// For async methods – use ThrowsAsync
// var ex = await Assert.ThrowsAsync<ArgumentException>(
//     async () => await service.SaveAsync(null));
```

Tip: If you `Assert.Throws<Exception>` but the code throws `DivideByZeroException`, the test **passes** because `DivideByZeroException : Exception`. Be specific – use the exact exception type.

Step 5 – Shared Context: `IClassFixture` + `IDisposable`

When multiple tests share the same expensive object, use `IClassFixture<T>`.

```
// 📁 Day1.OrderSystem.Tests/OrderServiceTests.cs

// Fixture – created ONCE, shared across all tests in the class
public class OrderServiceFixture
{
    public OrderService Service { get; } = new OrderService();
}
```

```
// IClassFixture<T> tells XUnit to inject the fixture via constructor
public class OrderServiceTests : IClassFixture<OrderServiceFixture>, IDis
{
    private readonly OrderService _service;

    // Constructor runs before EACH test – inject the shared fixture here
    public OrderServiceTests(OrderServiceFixture fixture)
    {
        _service = fixture.Service;
    }

    // Dispose runs after EACH test – clean up per-test resources here
    public void Dispose() { /* release per-test resources */ }

    [Fact]
    public void PlaceOrder_ValidCart_ReturnsConfirmedOrder()
    {
        var cart = new Cart();
        cart.Add(new CartItem("Laptop Stand", 49.99m, 1));
        var customer = new Customer("Alice", CustomerTier.Regular);

        var order = _service.PlaceOrder(cart, customer);

        Assert.Equal(OrderStatus.Confirmed, order.Status);
    }
}
```

Step 6 – [MemberData] and [ClassData]

For complex or typed test data that can't fit in [InlineData] constants.

```
// 📁 Day1.OrderSystem.Tests/TestData/DiscountMemberData.cs

// [MemberData] – static property on any class
public static class DiscountMemberData
{
    public static IEnumerable<object[]> Cases =>
    [
        [new Customer("Alice", CustomerTier.Regular), 200m, 0m ],
        [new Customer("Bob", CustomerTier.Silver), 200m, 20m ],
    ]
}
```

```

        [new Customer("Carol", CustomerTier.Gold), 200m, 30m ],
    ];
}

// 📁 Day1.OrderSystem.Tests/TestData/CancelOrderClassData.cs

// [ClassData] - IEnumerable<object[]> class
public class CancelOrderClassData : IEnumerable<object[]>
{
    public IEnumerator<object[]> GetEnumerator()
    {
        yield return [OrderStatus.Pending, true ];
        yield return [OrderStatus.Confirmed, true ];
        yield return [OrderStatus.Shipped, false];
    }
    IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
}

// Usage in test class:
[Theory]
[MemberData(nameof(DiscountMemberData.Cases), MemberType = typeof(Discour
public void CalculateDiscount_MemberData(Customer customer, decimal total
{ Assert.Equal(expected, _service.CalculateDiscount(customer, total)); }

[Theory]
[ClassData(typeof(CancelOrderClassData))]
public void CancelOrder_ClassData(OrderStatus status, bool expectedResult
{ var order = BuildOrder(status); Assert.Equal(expectedResult, _service.C

```

📁 Full source: [Day1.OrderSystem.Tests/TestData/](#)

Step 7 – Introducing Fluent Assertions

The **same assertions** as Steps 2-4, rewritten using FluentAssertions.

Notice how the code reads left-to-right like a sentence.

```

// 📁 Day1.OrderSystem.Tests/OrderServiceTests_FA.cs
using FluentAssertions; // ← only import needed

// — Before (native Assert) —————
Assert.NotNull(order);
Assert.Equal(OrderStatus.Confirmed, order.Status);

```

```

Assert.Single(order.Items);
Assert.Equal(15m, order.Discount);

// — After (FluentAssertions) —————
order.Should().NotBeNull();
order.Status.Should().Be(OrderStatus.Confirmed);
order.Items.Should().HaveCount(1);
order.Discount.Should().Be(15m);

// — Exception: Before —————
var ex = Assert.Throws<InvalidOperationException>(() => service.PlaceOrder());
Assert.Contains("minimum", ex.Message, StringComparison.OrdinalIgnoreCase);

// — Exception: After —————
Action act = () => service.PlaceOrder(cart, customer);
act.Should()
    .Throw<InvalidOperationException>()
    .WithMessage("*minimum*"); // * is a wildcard

```

The error messages from FluentAssertions are significantly more descriptive.
Run both test classes side by side – both pass; FA version reads better.

Step 8 – FA Advanced: Chaining, Collections & Object Graphs

Fluent Assertions shines with complex objects and collections.

```

// 📁 Day1.OrderSystem.Tests/OrderServiceTests_FA.cs

// — Chaining multiple assertions with .And —————
order.Discount.Should().Be(15m).And.BePositive();

bool result = _service.CancelOrder(order);
result.Should().BeFalse("shipped orders cannot be cancelled");
order.Status.Should().Be(OrderStatus.Shipped);

// — Collections —————
order.Items.Should()
    .HaveCount(2)
    .And.NotContainNulls()
    .And.Contain(i => i.ProductName == "Laptop Stand");

```

```
// — Deep object graph equality —————  
var expected = new { Status = OrderStatus.Confirmed, Discount = 15m };  
order.Should().BeEquivalentTo(expected,  
    opts => opts.ExcludingMissingMembers()); // compare overlapping prop  
  
// — Strings —————  
order.Customer.Name.Should()  
    .NotBeNullOrWhiteSpace()  
    .And.StartWith("A");
```

📁 Full FA demo: `Day1.OrderSystem.Tests/OrderServiceTests_FA.cs`